Neura

Integrated E-Learning Platform

Moataz Mohamed Mohamed **Mahmoud Emad Nouh**

Amany El-Sayed Kamal **Marwa Mohamed Hassan**

Yousef Salah Mohamed **Mahmoud Emad El-Din**

Supervised by

Dr. Ahmed Salama

2025 / 2026

Acknowledgments

We would like to express our sincere gratitude to all those who have supported us throughout the course of this project. In particular, we extend our deepest thanks to Dr. Ahmed Salama for their invaluable guidance, advice, and encouragement throughout the project's development.

Abstract

This project introduces **Neura**, an innovative integrated e-learning platform that synthesizes five specialized technological domains into a single, unified, and customizable system for modern technical education. Neura addresses the critical "integration gap" in the current e-learning market by providing native implementations of: adaptive practice powered by Deep Knowledge Tracing (DKT), AI-driven tutoring using Retrieval-Augmented Generation (RAG), secure examination with AI-powered proctoring, source code plagiarism detection using AST-based machine learning, and sandboxed code execution through containerized online judge systems.

Unlike existing platforms that require institutions to "bolt together" multiple disparate systems—such as Moodle, Respondus, MOSS, and beecrowd—Neura provides all five functions within a single, secure, and scalable architecture. The platform is built on a modern technology stack: .NET 9 backend, React (v19) frontend, Microsoft SQL Server database, and specialized Python services for AI components. Each subsystem is grounded in state-of-the-art research, from DKT models for adaptive learning to RAG architectures for educational chatbots, ensuring both academic rigor and practical effectiveness.

Neura's architectural synthesis represents a novel contribution to e-learning technology, bridging the gap between general-purpose MOOCs and niche, single-feature tools. The platform enables institutions to deliver high-integrity, customizable technical education without the complexity and security concerns of managing multiple vendor relationships and data silos. Future work will focus on advanced personalization, expanded AI capabilities, and enhanced community features.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Overview

In an era where technology continuously reshapes how we learn, this project introduces an advanced educational platform designed to bridge the gap between conventional education and modern, data-driven approaches. By integrating intelligent analytics, interactive content, and strong community engagement, the platform aims to deliver a personalized and adaptive learning experience. The system provides a unified digital environment where learners can access structured learning paths, participate in interactive activities, and monitor their progress through detailed performance insights. Through AI-powered analytics and smart recommendations, each learner receives guidance and feedback tailored to their unique pace, preferences, and goals.

At its initial stage, the project focuses on building an engaging and collaborative learning space that connects learners, mentors, and the broader community through interactive challenges and guided activities. Over time, it aims to expand into a comprehensive educational ecosystem that can serve diverse learning domains—from competitive problem-solving and high school preparation to professional skill development and specialized courses. This gradual evolution will enable the platform to support a variety of learning paths while maintaining its core focus on personalization, accessibility, and meaningful engagement. As education continues to evolve in the digital age, this project aspires to stand as a foundation for meaningful, efficient, and adaptive learning experiences that inspire continuous growth and curiosity.

1.2 Problem Statement

The evolution of digital education has introduced remarkable opportunities for learning, yet it has also surfaced fundamental challenges that can hinder a truly effective educational experience. A significant gap exists in fostering a genuine sense of community and human connection. Learning in a digital space can become an isolating journey, where the lack of spontaneous interaction between students and instructors, as well as among peers, creates barriers to collaborative problem-solving and shared growth. This absence of a supportive network often leads to decreased motivation and makes it difficult for students to seek timely help.

Furthermore, ensuring the integrity and fairness of academic assessments in a remote setting remains a critical hurdle. Without robust and reliable supervision, the credibility of online examinations is often compromised. This not only casts doubt on the validity of the results but also impacts the perceived value of the entire learning process, making it difficult to accurately measure and certify a student's mastery of a subject.

Finally, the promise of personalized education is seldom fully realized. A truly adaptive experience requires a deep understanding of each learner's individual progress, strengths, and weaknesses. The difficulty in identifying and catering to different student levels leads to a standardized approach that may overwhelm struggling students or fail to challenge high-achievers. This, combined with a need for simple, intuitive design, highlights a demand for an educational framework that is not just a repository of content, but a dynamic, responsive, and engaging environment.

1.3 Proposed Solution

Our educational platform offers an integrated solution that transforms the digital learning experience by combining modern technologies with fundamental educational needs. The platform is designed to address key gaps in e-learning through a unified and comprehensive learning environment. The platform creates smart learning pathways that dynamically adapt to each student's performance, continuously analyzing educational data to deliver personalized content and recommendations tailored to each learner's level and individual needs.

To ensure assessment credibility, the platform integrates an advanced AI-powered monitoring system that maintains academic integrity by tracking student behavior during exams and detecting any cheating attempts, while

implementing strict security measures that guarantee evaluation fairness. The platform also builds an interactive educational community that connects students and teachers through organized communication channels, providing a collaborative environment that eliminates isolation in digital education and enhances cooperative learning. The interface is designed to be intuitive and user-friendly, focusing on simplifying the technical experience and making all features smoothly accessible to users regardless of their technical background. Through this integration, the platform creates a comprehensive educational system that combines personalized learning, secure assessment, and interactive community in one cohesive environment.

1.4 Project Scope

1.4.1 Scope Overview

This project encompasses the end-to-end development of a scalable educational platform designed to facilitate interactive learning, ensure academic integrity, and deliver personalized educational experiences. The platform will integrate core learning management, AI-proctored assessments, and data-driven analytics within a unified environment, establishing a foundation for future expansion into diverse learning domains and institutional adoption.

1.4.2 In-Scope

- **Core Learning Management System (LMS):** A full-featured system for course enrollment, content delivery, and progress tracking. This includes support for diverse learning materials (videos, PDFs, interactive content), structured learning paths, and automated progress monitoring to create a seamless educational experience.
- **Secure Examination & Proctoring System:** An AI-powered assessment environment that ensures academic integrity through live webcam monitoring, behavioral analysis, and comprehensive cheating prevention mechanisms. The system will enforce strict exam protocols including browser lockdown and activity monitoring.
- **Personalized Learning Analytics:** Advanced data processing capabilities that transform user performance data into actionable insights. This includes personalized learning path recommendations, progress visualization, and detailed reporting for both students and educators to enable targeted interventions.

- **Community & Communication Hub:** Foundational tools for structured interaction between students, teachers, and assistants. This includes real-time messaging, organized discussion channels, and integrated notifications, designed to foster a supportive learning community and reduce the isolation of online education.

1.4.3 Out of Scope

- **Mobile Application Development:** Native iOS or Android mobile applications are excluded from the initial release, though the web platform will be optimized for mobile browsers.
- **Advanced Gamification and Social Features:** Complex game-like elements such as points, badges, leaderboards, and advanced social networking features are reserved for future iterations to keep the initial focus on core learning and assessment.
- **Third-Party Platform Integrations:** Direct integrations with external systems such as existing Learning Management Systems (L.g., Google Classroom, Moodle) or enterprise software are not included in the initial scope.
- **AI-Powered Virtual Tutor:** An AI-driven assistant that provides real-time, conversational help or personalized tutoring sessions is excluded from the initial project scope.
- **Offline Access Mode:** The ability for users to download content and use the platform without an active internet connection is not a feature of the initial release.

1.4.4 Assumptions

- Users have access to internet connectivity and compatible browsers to interact with the web application.
- High-performance hardware or cloud services will be required to handle complex video processing tasks.
- Educators will provide quality content for the platform's learning modules.
- Users will adhere to academic integrity policies during assessments.

1.4.5 Constraints

- The AI-proctoring system's accuracy is dependent on training data quality.
- Platform performance is tied to computational resources for heavy processing tasks.
- Data privacy regulations require robust encryption and secure data handling.
- Browser compatibility is limited to modern, evergreen browsers.

1.5 Project Limitations

1.5.1 Technical Limitations

- The AI-proctoring system's accuracy may not reach 100% in detecting all cheating attempts, especially in complex scenarios.
- Platform performance for video processing and real-time analytics depends heavily on available computational resources.
- Full functionality is guaranteed only on modern browsers (Chrome, Firefox, Safari, Edge) due to dependency on latest web APIs.

1.5.2 Resource Limitations

- Initial deployment will focus on core features, with advanced community tools postponed for future iterations.
- The recommendation engine requires substantial user data to provide highly accurate personalized suggestions.
- Mobile experience will be limited to responsive web design without native app features.

1.5.3 Implementation Limitations

- Data privacy compliance may restrict certain data collection and processing methods.
- The adaptive learning system requires continuous refinement based on user feedback and behavior patterns.

- Integration capabilities with institutional systems will be limited in the initial release.

1.5.4 Operational Limitations

- User adoption and community growth depend on effective onboarding and engagement strategies.
- The effectiveness of collaborative features relies on a critical mass of active users.
- Maintenance and updates may require temporary service interruptions during initial deployment phases.

1.6 Project Objectives

The primary objectives of this project are designed to directly address the identified problems in digital education:

1.6.1 Enhance Learning Engagement

- Develop an interactive community platform that reduces isolation through structured peer-to-peer and student-mentor interactions.
- Implement organized communication channels that maintain academic focus while fostering collaborative learning.
- Create intuitive user interfaces that minimize technical barriers and maximize accessibility.

1.6.2 Ensure Academic Integrity

- Build a robust AI-proctoring system capable of detecting cheating attempts through computer vision analysis.
- Implement comprehensive exam security measures including browser lockdown and activity monitoring.
- Establish a trustworthy assessment environment that maintains the credibility of online evaluations.

1.6.3 Deliver Personalized Learning

- Create adaptive learning pathways that respond to individual student performance and progress.
- Develop intelligent recommendation systems for content and practice materials.
- Provide detailed analytics and reporting tools for both students and educators.

1.6.4 Establish Scalable Foundation

- Design a modular architecture supporting future expansion into new educational domains.
- Ensure platform performance under increasing user loads and feature additions.
- Maintain flexibility for integration with emerging educational technologies and methodologies.

These objectives collectively aim to create an educational environment that is not only effective in knowledge delivery but also supportive of student growth, trustworthy in evaluation, and adaptable to future educational needs.

Chapter 2

Related Work

2.1 Introduction

This chapter situates the "Neura" project within the broader context of the modern e-learning technology landscape. The platform's innovation is not the invention of a novel, singular technology, but rather the **architectural synthesis** of five distinct, high-integrity, and specialized technological domains into a single, unified, and customizable platform. Current platforms are typically strong in one or two of these domains but fail to provide a cohesive solution.

The analysis herein is structured in two primary sections. First, an examination of current commercial and open-source platforms is conducted to identify the "integration gap" in the existing market. Second, a review of the specialized academic and technical literature is presented for each of Neura's five core functional pillars: (1) adaptive practice, (2) AI-driven tutoring, (3) secure examination, (4) source code plagiarism detection, and (5) sandboxed code execution. This review establishes the established, state-of-the-art foundations upon which Neura is built.

2.2 Analysis of Commercial E-Learning Platforms

The contemporary e-learning market is highly fragmented, forcing institutions to "bolt together" multiple disparate systems to achieve the functionality Neura provides natively. Existing platforms can be broadly classified into two categories:

1. **Massive Open Online Course (MOOC) Platforms:** These platforms, such as Coursera and edX, are masters of content distribution and scale but are architecturally closed and offer weak, non-customizable tooling for high-integrity assessment.[1, 2] While they provide verified

certificates and may integrate with third-party proctoring services [1], their native coding environments are typically basic text boxes, lacking the secure sandboxing of a true online judge or the sophisticated, AST-based analysis of a dedicated code plagiarism detector.

2. **Specialized, Single-Purpose Platforms:** This category includes tools that perform one function exceptionally well. For example, **beecrowd** (formerly URI Online Judge) is a widely used platform in academia for competitive programming and serves as an excellent online judge.[3] Similarly, **Respondus LockDown Browser** is a market leader in secure browsers for exams [4], and **MOSS (Measure of Software Similarity)** is a long-standing tool for code plagiarism detection.[5] The fundamental limitation is that these are standalone products, requiring institutions to manage separate contracts, integrations, and data silos.

3. **Open-Source Learning Management Systems (LMS):** Platforms like **Moodle** represent a third-point solution, offering customization through a vast plugin ecosystem.[6] However, this modularity often introduces complexity and security concerns. An institution would need to find, vet, and maintain separate plugins for proctoring [6, 7], secure exam windows [7], and code evaluation. The community has long sought robust online judge integration for Moodle [8], highlighting the difficulty. Furthermore, some modules may have significant security vulnerabilities, such as a described exam module with "zero authentication and authorization".[10]

This analysis reveals a clear market gap. A computer science department, for example, must currently license Moodle [6], integrate a proctoring service like Respondus [4], manually run submissions through MOSS [5], and subscribe to a separate service like beecrowd [3] for assignments. Neura's value proposition is the elimination of this fragmented and costly integration by providing all five functions in a single, secure, and unified platform. This "integration gap" is visualized in Table 2.1.

2.3 Literature Review of Specialized E-Learning Subsystems

Neura's architecture is grounded in established computer science research. Each of its core features represents a domain of academic study.

Table 2.1: Feature Matrix of Competing E-Learning Platforms

Platform	Adaptive Practice	AI Tutor (RAG)	Secure Exam (AI Proctored)	Code Plagiarism (AST-based)	Online Judge (Sandboxed)
Neura (Proposed)	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**
Coursera[1]	Partial	No	Yes (3rd party)	No	Partial (basic)
edX[1]	Partial	No	Yes (3rd party)	No	Partial (basic)
Moodle[2]	Plugin-dependent	Plugin-dependent	Requires plugins	Requires MOSS/JPlag	Plugin-dependent
beecrowd[3]	No	No	No	No	**Yes**
HackerRank	No	No	No (recruitment)	Yes	**Yes**

2.3.1 Advancements in Adaptive Learning and Knowledge Tracing

Neura's adaptive practice feature is an implementation of **Computerized Adaptive Testing (CAT)**.[9, 10] CAT is a testing paradigm that dynamically adjusts the difficulty of questions presented to a student based on their real-time performance.[10, 11] This approach provides a more accurate and efficient assessment of student proficiency compared to static tests.[10]

The engine that powers CAT is a **Knowledge Tracing (KT)** model.[12, 13] KT is a task that aims to model a student's "evolving knowledge state" as they interact with a series of practice problems.[12, 14, 15]

- **Traditional Models:** Early models, such as Bayesian Knowledge Tracing (BKT), modeled knowledge as a binary, latent variable (known or unknown).
- **Modern Models:** The field has advanced significantly with the application of deep learning. **Deep Knowledge Tracing (DKT)** [12] utilizes Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks to process the *sequence* of student interactions (e.g., `[question_id, correct_boolean]`). This sequential modeling allows DKT to capture the nuances of learning and forgetting over time, providing a more accurate prediction of performance on future exercises.[12]

Neura's adaptive engine is based on this modern DKT approach, allowing it to recommend the optimal question to maximize a student's learning

efficiency.

2.3.2 AI-Driven Educational Chatbots (RAG)

The "AI Chatbot Teacher" moves beyond simple, scripted responses. The primary challenge with using foundational Large Language Models (LLMs) in education is their propensity to "hallucinate" or provide plausible-sounding but incorrect information, and their lack of specific knowledge about a given course's content.[16]

The state-of-the-art solution to this problem is **Retrieval-Augmented Generation (RAG)**.[16] A recent survey of RAG chatbots in education confirms their effectiveness and growing adoption.[17] The RAG architecture fundamentally "grounds" the LLM by following a two-step process:

1. **Retrieve:** When a student asks a question, the system first vectorizes the query and searches a trusted knowledge base (i.e., the course's PDFs, lecture transcripts, and materials) for the most relevant passages of text.[18]
2. **Augment:** The system then passes *both* the original question and this retrieved, "grounded" context to the LLM, instructing it to formulate an answer *only* using the provided information.[16, 19]

This RAG model ensures the chatbot's answers are faithful to the course material, prevents hallucination, and provides a safe, reliable study assistant that students can use without the "apprehension" of exposing their lack of knowledge to peers or instructors.[17]

2.3.3 Architectures for Secure Online Examination

Maintaining academic integrity in remote exams is a multi-layered problem, as documented in surveys on the topic.[20, 21] Neura's architecture implements a state-of-the-art, two-layer solution.

- **Layer 1: Client-Side Lockdown:** The first layer involves restricting the student's testing environment. This is accomplished using a **lockdown browser** application, such as the widely used Respondus LockDown Browser.[4] This custom browser prevents students from opening new tabs, taking screenshots, copy-pasting, or accessing other applications during an exam.[22]

- **Layer 2: AI-Powered Proctoring:** The second layer is a sophisticated multimedia analytics system that monitors the student for behavioral anomalies.[23] A survey of proctoring system architectures [24] and case studies, such as the "Proctor SU" system [20], reveal a common technical design. This design, which Neura adopts, involves:

    1. **Stream Ingestion:** Capturing the student's webcam and desktop feeds using **WebRTC** technology.[25, 26]
    2. **AI Analysis Pipeline:** Processing the streams in real-time using a pipeline of machine learning models.[26, 27] This typically includes:
        - **Face Detection/Recognition (CNN):** To ensure the registered student is present and not an imposter.[27, 28]
        - **Object Detection (e.g., YOLOv3):** To detect forbidden items, such as a mobile phone.[20]
        - **Gaze/Head Pose Estimation:** To detect if the student is consistently looking off-screen.
        - **Audio Analysis:** To detect if another person is speaking in the room.
        - **Active Window Detection:** To monitor for forbidden applications on the desktop feed.[20]

This multi-model, AI-driven approach provides a robust and scalable method for ensuring academic integrity.[20]

2.3.4 Source Code Plagiarism Detection

Detecting plagiarism in programming assignments is significantly more complex than in text, as students can easily rename variables, reorder functions, or add spurious comments to disguise copying.

- **Traditional Tools:** The foundational tools in this domain are **MOSS (Measure of Software Similarity)** [5] and **JPlag**.[29, 30, 31] These systems are far more advanced than simple text diffs. Their core algorithm, as described for JPlag [31], involves:

    1. **Tokenization:** The source code is parsed into a stream of abstract tokens (e.g., `int x = 10;` becomes `TYPE_INT VAR ASSIGN INT_LITERAL SEMICOLON`). This makes the system immune to changes in variable names or whitespace.

2. **String Matching:** The system applies a **Greedy String Tiling** algorithm [31] to find the longest common token sequences between all pairs of submissions.

- **Modern Techniques & Limitations:** While effective, these tools can be defeated by more sophisticated obfuscation, such as refactoring code structure.[32] Modern research, and the approach adopted by Neura, enhances this process by:

  1. **Abstract Syntax Trees (ASTs):** Parsing the code into an AST—a tree representation of the code's logical structure—and comparing the *trees* themselves.[33, 34] This allows the system to identify similarity even if functions are reordered or nested differently.
  2. **Machine Learning:** Employing ML models [35] (e.g., Gradient Boosting, Extra Trees [36]) trained on code feature vectors (e.g., TF-IDF of tokens, AST node counts) to produce a similarity score that is robust to a wide range of obfuscation techniques.[37]

Neura's plagiarism model implements this modern, hybrid AST/ML approach to provide a higher degree of detection accuracy.

2.3.5 Online Judge Systems and Secure Code Execution

An **Online Judge (OJ)** system is an essential tool for computer science education, enabling the automatic grading of programming assignments by compiling and executing student code against a set of predefined test cases.[38]

As identified by a comprehensive survey of OJ systems [39], the single most critical design challenge is **security**.[39] The system must execute untrusted, user-submitted code without allowing that code to compromise the host server. A malicious submission could contain a fork bomb (e.g., `while(true) { fork(); }`), attempt to read server file systems, or initiate network attacks.

The universally accepted solution is **sandboxing**.[40]

- **Modern Architecture (Containerization):** While early systems used virtual machines or complex `chroot` jails, the modern, scalable, and secure architecture pioneered by open-source systems like **Judge0** [41, 42, 43] is based on lightweight **containerization**.[44, 45]

- **Execution Flow:** In this model, each code submission triggers the creation of a new, ephemeral **Docker container**.[45, 46] This container

is a minimal, isolated environment with no access to the host system and is given strict resource limits (e.g., 2 seconds of CPU time, 256 MB of RAM).[41] The student's code is compiled and executed *inside* this sandbox.[40] The system captures its output, compares it to the expected result, and then immediately *destroys* the container.

Neura's Online Judge directly implements this robust, secure, and scalable container-based architecture.

2.4 Identifying the Gap: Neura's Novel Contribution

The related work review confirms two key findings:

1. **The Commercial Gap:** The major e-learning platforms (Table 2.1) lack the deep, integrated, and specialized tools required for high-integrity, customizable technical education. Institutions are left to use a fragile and expensive patchwork of single-purpose products.
2. **The Academic Gap:** The academic literature is highly *siloed*. Research focuses intensely on one domain at a time: secure proctoring [20], RAG chatbots [16], plagiarism [29], or online judges.[39]

The novelty of the Neura project is the **architectural synthesis** of all five. It is the first platform designed to *unify* these disparate, high-complexity systems—which are normally standalone products or isolated research projects—into a single, cohesive, and customizable platform. Neura bridges the gap between general-purpose MOOCs and niche, single-feature tools, creating a new category of integrated, high-integrity platforms for modern technical education.

Chapter 3

Technology Stack

3.1 Introduction

The technology stack for the "Neura" platform was selected to meet foundational requirements of scalability, security, performance, and maintainability. Given the platform's nature as an integrated system—combining real-time services, high-integrity assessment, and data-intensive AI models—our architecture is a modern, decoupled system. It consists of a high-performance backend, a reactive web frontend, and a suite of specialized AI and infrastructure services.

3.2 Core Platform Architecture

The central platform handles user management, content delivery, and course administration.

3.2.1 Backend: .NET 9

The entire backend API and core business logic are built on the .NET 9 framework, specifically using ASP.NET Core.

- **Rationale:** .NET 9 offers exceptional performance, robust security features, and a unified development model. Its asynchronous processing capabilities are ideal for handling a high volume of concurrent users.
- **Language:** We utilize **C# 13**, a modern, type-safe, object-oriented language.
- **API:** A RESTful API serves as the primary communication layer between the frontend and backend, using JSON as the standard data interchange format.

- **ORM:** We use **Entity Framework Core 9 (EF Core)** as the Object-Relational Mapper (ORM) to manage database interactions, enabling rapid and secure data access.

3.2.2 Frontend: React (v19)

The user interface is a Single Page Application (SPA) built using React.

- **Rationale:** React's component-based architecture allows for the creation of a complex, reusable, and maintainable UI. Its Virtual DOM ensures a fast and responsive user experience.
- **Features:** We leverage modern React features, including Hooks for state management, React Context for global state, and the new capabilities introduced in React 19 to optimize rendering.
- **Styling:** Tailwind CSS is used for a utility-first styling approach, allowing for rapid prototyping and custom-designed components without leaving the HTML.

3.2.3 Database: Microsoft SQL Server

Microsoft SQL Server is our primary relational database management system (RDBMS).

- **Rationale:** As a .NET project, SQL Server provides seamless integration, enterprise-grade security, and high performance. It is designed for high-concurrency applications and complex data models.

3.2.4 Real-time Communication: SignalR

For the community and chat features, real-time bidirectional communication is managed by **ASP.NET Core SignalR**.

- **Rationale:** SignalR is deeply integrated into the .NET ecosystem, simplifying the process of adding real-time web functionality. It handles connection management and scales efficiently.

3.3 Specialized Subsystems

The core features of Neura are implemented as distinct services, which may be co-located or deployed independently.

3.3.1 Adaptive Recommendation Engine

This service provides the logic for recommending appropriate practice questions.

- **Technology:** Python
- **Models:** The engine is built using **TensorFlow** and **Keras** to implement a Deep Knowledge Tracing (DKT) model. This neural network analyzes a student's sequence of past answers to model their knowledge state and predict future performance.

3.3.2 AI Chatbot Teacher (RAG)

The chatbot provides contextual, course-aware assistance to students.

- **Architecture:** Retrieval-Augmented Generation (RAG).
- **Technology:** Python, **LangChain** (or a similar orchestration framework), and a vector database (e.g., **FAISS** or **ChromaDB**).
- **Process:** When a student asks a question, the system vectorizes the query, retrieves relevant chunks of text from the course materials (e.g., PDFs, lecture notes) stored in the vector database, and passes this context to an LLM API to generate a "grounded" and factual answer.

3.3.3 Online Judge & Code Execution

This subsystem is responsible for securely compiling and executing student-submitted code.

- **Technology: Docker** and a custom microservice built in Python (using **Flask** or **FastAPI**).
- **Security:** This is a critical security component. We follow the architectural principles of systems like **Judge0**. Each code submission is executed within a new, ephemeral, and isolated Docker container with strict resource limits (CPU time, memory, network access) to prevent malicious code from affecting the host system.

3.3.4 Source Code Plagiarism Detection

This model analyzes all code submissions for a given assignment to detect similarity.

- **Technology:** Python
- **Models:** We employ a hybrid approach. First, code is parsed into an **Abstract Syntax Tree (AST)** using Python's `ast` module. Feature vectors (e.g., node counts, TF-IDF of tokens) are extracted from the ASTs and then fed into **scikit-learn** machine learning models (e.g., Gradient Boosting) to compute a robust similarity score.

3.3.5 Secure Examination & AI Proctoring

This system combines client-side lockdown with real-time AI monitoring.

- **Client-Side:** A custom lockdown browser (built with **Electron.js**) or integration with existing lockdown browsers.
- **Media Streaming:** **WebRTC** is used to capture and stream the student's webcam and desktop feeds to the proctoring service.
- **AI Analysis:** A Python service using **OpenCV** and **TensorFlow** processes the video feeds in real-time to detect anomalies, such as multiple faces, mobile phone presence, or gaze deviation.

3.4 Summary of Technologies

The complete technology stack is summarized in Table 3.1.

Table 3.1: Summary of Technology Stack per Feature

Component / Feature	Primary Technology	Rationale
Core Backend (API, Logic)	.NET 9 (C#)	Performance, Security, Unified Stack
Core Frontend (UI)	React (v19)	Component-Based, Responsive UI
Database	MS SQL Server	RDBMS, .NET Integration, Scalabili
Real-time Chat/Community	SignalR	Integrated Real-time Communication
Adaptive Practice	Python, TensorFlow (DKT)	Sequential Data Modeling
AI Chatbot Teacher	Python, RAG, Vector DB	Context-Aware, Grounded Answers
Online Judge	Docker, Python	Secure, Sandboxed Code Execution
Plagiarism Detection	Python, AST, scikit-learn	Robust Similarity Analysis
Secure Exam Proctoring	WebRTC, OpenCV, TensorFlow	Real-time Anomaly Detection

Chapter 4

System Analysis and Design

4.1 Introduction

This chapter provides a comprehensive analysis of the educational platform's architecture, detailing its structural components, data flow, and user interactions. Through various modeling techniques, we break down the system's core functionalities and processes to ensure a robust foundation for implementation. The analysis employs multiple diagrammatic approaches to illustrate different perspectives of the system, from high-level ecosystem interactions to detailed process workflows. The analytical framework includes:

- **Context Diagram:** Presenting the platform's ecosystem and its interactions with external entities and user roles.
- **Data Flow Diagram (DFD):** Illustrating the movement and transformation of data across system processes.
- **Activity Diagrams:** Modeling the progression of business processes and user workflows within the platform.
- **Class Diagram:** Defining the static structure of the system through entities, their attributes, and relationships.
- **Sequence Diagram:** Depicting the chronological flow of interactions between system components and external services.
- **Use Case Diagram:** Offering a high-level representation of system functionalities and interactions between actors.
- **Use Case Analysis:** Providing a detailed examination of use case steps, actors, and conditions.

Collectively, these diagrams establish a clear blueprint for system development, ensuring all functional requirements are properly addressed while maintaining scalability and user experience excellence.

4.2 System Requirements

This section outlines the core functional and non-functional requirements for the educational platform.

4.2.1 Functional Requirements

- Users can register, log in, and manage their profiles with role-based access (Student, Mentor, Admin).
- Students can browse courses, enroll in available courses, and access learning materials (videos, PDFs, problem sets).
- The system supports submission of solutions to problems, with automatic evaluation and verdict delivery.
- Mentors can create and manage course content, monitor student progress, and facilitate group interactions.
- Admins can manage users, assign roles, oversee site settings, and generate system-wide reports.
- The platform provides personalized learning recommendations based on student performance and progress analytics.
- The system integrates with external judges for real-time code evaluation and plagiarism detection.

4.2.2 Non-Functional Requirements

- The system must implement robust data encryption for all sensitive user information, particularly national and college IDs, ensuring secure storage and transmission.
- Access control must be enforced through role-based permissions for administrative dashboards, utilizing JWT tokens and HTTPS for secure API communication.

- The platform must maintain high availability through asynchronous processing for code evaluation and real-time features like chat and notifications.
- System performance must be monitored through key metrics including user engagement, submission pass rates, and course completion rates.
- Multi-channel notifications must be implemented for critical alerts, distributed via in-app messages, email, and integrated platforms like Discord and Telegram.
- The architecture must support horizontal scaling to accommodate growing user loads while maintaining response times under 2 seconds for core features.
- The platform must implement automated spam detection and rate limiting to ensure system stability and prevent resource exhaustion.

4.3 Context Diagram

The context diagram provides the highest-level view of the E-Learning platform, representing it as a single process and illustrating its interactions with all major external entities.

4.4 Data Flow Diagram

A Data Flow Diagram (DFD) illustrates how the E-Linker platform processes information, focusing on the movement of data between processes, external entities, and data stores. It provides a clear overview of how data is transformed, where it is stored, and how it flows through the system to support various educational functions.

4.5 Use Case Diagram

The Use Case Diagram provides a complete visual representation of the system's functional scope and all authorized interactions for each primary actor. A use case diagram illustrates the platform's functionality from the user's perspective. It identifies all system users and demonstrates how they interact with the platform. The diagram visualizes relationships between different use cases and actors. It defines the boundary between the system and its external environment.

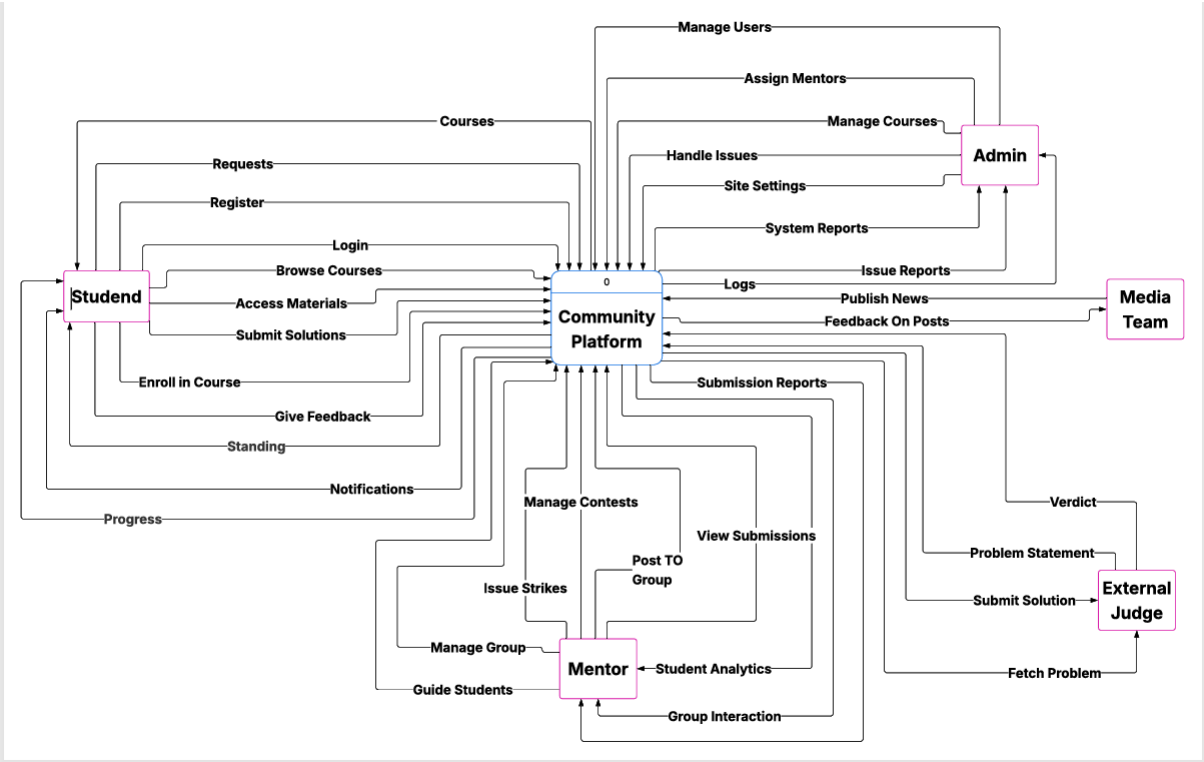


Figure 4.1: System Context Diagram.

4.6 Use Case Scenarios: Detailed System Flows

Detailed scenarios define the step-by-step interactions for core, complex, and specialized system processes.

4.6.1 UC 1: User Registration and Identity Verification

This scenario details the complete workflow for new user onboarding, incorporating multi-level verification to ensure platform integrity and compliance.

1. **Student** navigates to the registration page and provides essential credentials: username, full name, email, and password.
2. Optionally submits verification documents: phone number, college ID, or national ID.
3. **System** performs real-time validation checking for duplicate usernames/emails and data format compliance.
4. **Decision Point (Validation):**
    - If Validation Fails: System returns specific field-level error messages with correction guidance.

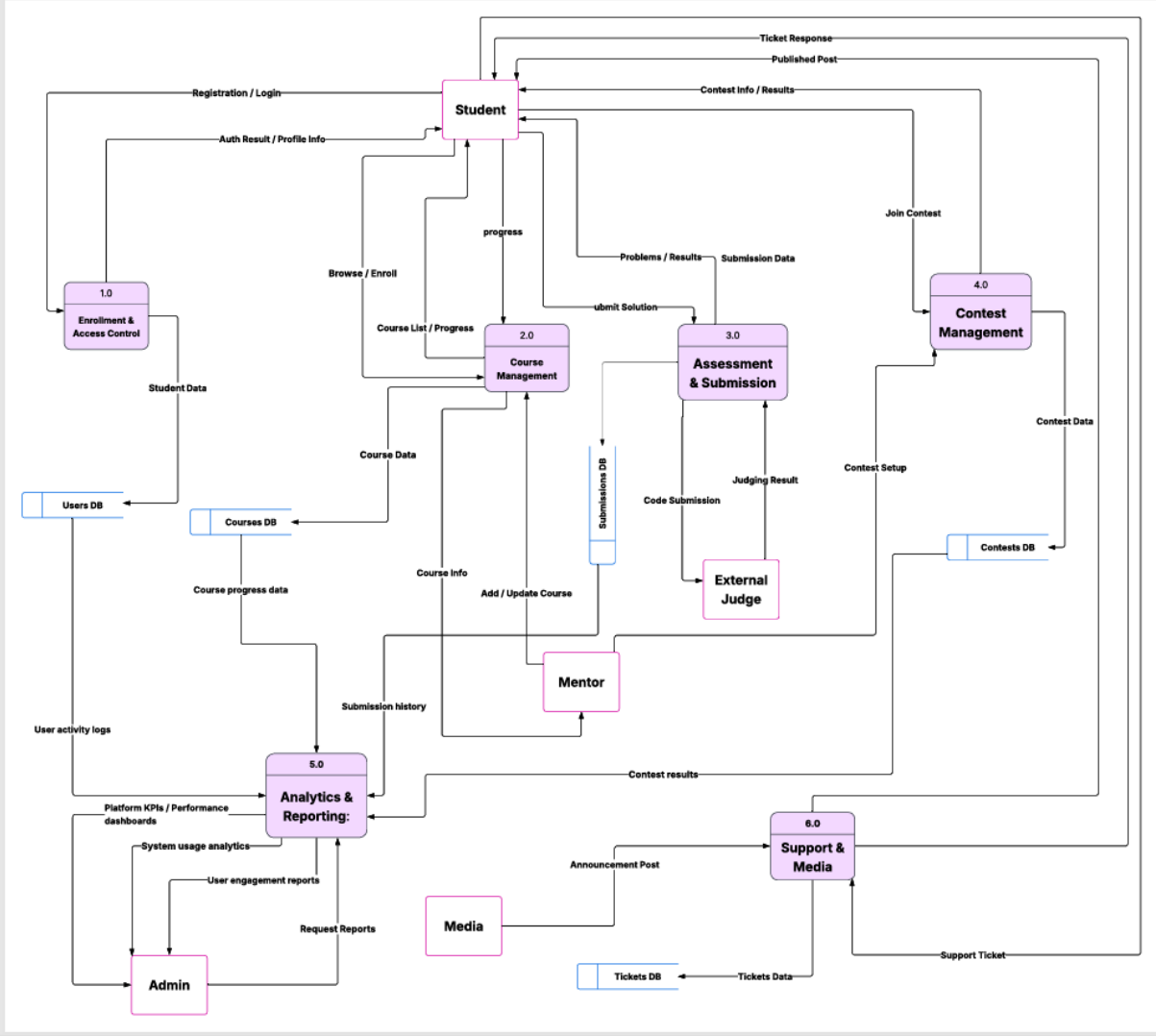


Figure 4.2: Level 1 Data Flow Diagram (DFD).

- If Validation Succeeds: System creates a temporary account with "pending verification" status and dispatches a verification email with a time-sensitive activation link.

5. For ID submissions: **System** routes documents to an encrypted Reviewer Dashboard queue for manual verification.

6. **Reviewer** conducts manual verification comparing submitted documents against user profile data.

7. **Decision Point (Verification Level):**

   - Basic Verification (email only): Grants access to public courses and community features.
   - Enhanced Verification (ID approved): Unlocks exam participation, contest access, and premium content.

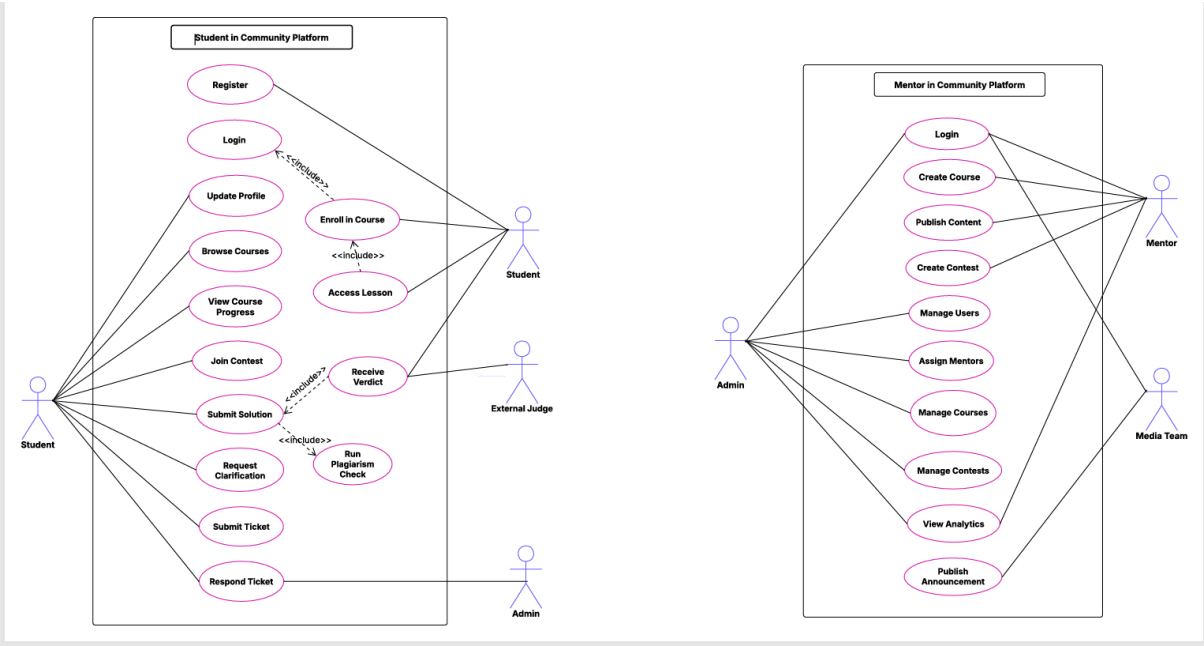


Figure 4.3: System Use Case Diagram.

8. **System** notifies the user of verification status and corresponding access privileges, generating a comprehensive audit trail of registration events.

4.6.2 UC 2 (Synthetic): Secure Exam Initiation and Proctoring

This scenario illustrates the multi-layered process for deploying high-integrity online exams, combining system controls and AI monitoring.

1. **Student** navigates to the exam page and clicks 'Start Secure Exam'.
2. **System** validates the student's environment by utilizing the JavaScript Visibility API to check for multiple open browser tabs or windows, blocking the start if unauthorized activity is detected.
3. **System** requests and verifies access to the student's Webcam and Microphone, initializing the Computer Vision components (OpenCV/MediaPipe).
4. **System** programmatically activates anti-cheating controls, specifically disabling native browser functions for copy/paste and preventing screen capture.
5. **Secure Exam Module** loads the exam content, ensuring it is decrypted only upon start and held in a protected local storage mechanism (Secure Caching/Encrypted IndexedDB) to prevent content leakage.

6. The **AI/CV Monitoring Service** begins real-time streaming and analysis of video/audio data, actively searching for behavioral anomalies that indicate cheating (e.g., unusual head movements, detection of a second person).

7. **Student** is presented with the exam content and begins the assessment under continuous surveillance.

4.6.3 UC 3 (Synthetic): Personalized Question Recommendation

This flow describes the data pipeline that powers the adaptive learning system.

1. **Student** navigates to the course practice area and selects 'Get Personalized Practice'.

2. **Core System** queries the Analytics DB (DFD Process 5.0) to retrieve the Student's complete performance metrics, including their current level, recent assessment scores, and detailed submission history.

3. The relevant data subset is transmitted to the dedicated **ML Recommendation Service** (running Python + scikit-learn).

4. The **ML Service** processes the student's profile against its models, using performance metrics (e.g., time-to-solve, tags of incorrectly answered questions) to generate a set of optimal Question IDs, potentially leveraging ML embeddings for precision.

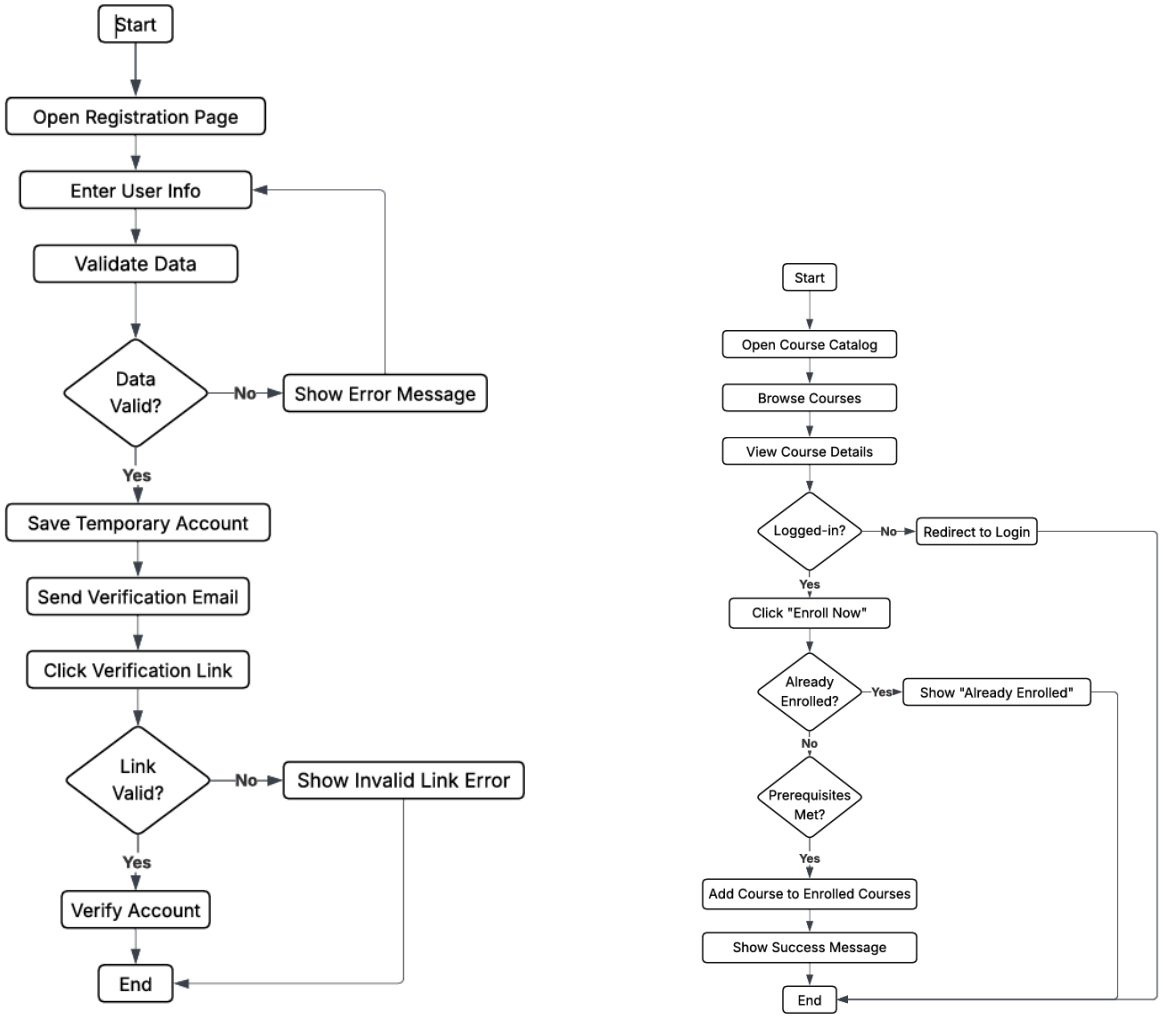
5. The **ML Service** returns the prioritized list of recommended Question IDs to the Core System.

6. **Core System** retrieves the full problem statements and associated metadata for the recommended questions from the database.

7. **Student** receives the practice set, which is dynamically tailored to address their specific identified knowledge gaps and weaknesses.

4.7 Activity Diagrams

Activity diagrams model the processes required to execute key functionalities, highlighting branching and parallel execution.



(a) User Registration.

(b) Course Enrollment.

Figure 4.4: Activity Diagrams for User Onboarding and Enrollment.

4.8 Sequence Diagram

Sequence Diagrams illustrate the dynamic interaction and message passing between system components, confirming the operational feasibility of decoupled and specialized features.

4.9 Class Diagram

The Class Diagram defines the definitive static structure of the E-learning system, detailing specialized classes and their encapsulated operations, directly implementing the platform's unique requirements.

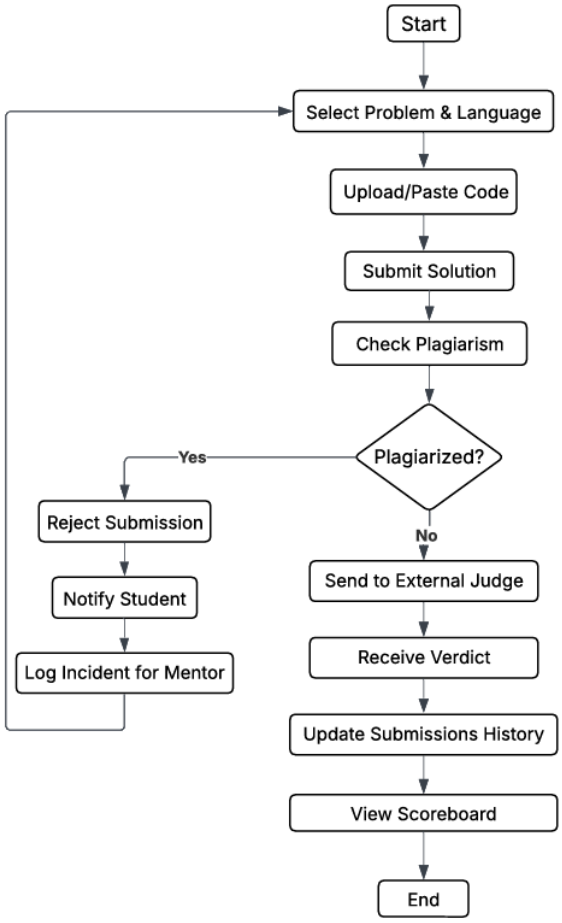


Figure 4.5: Activity Diagram for Code Submission.

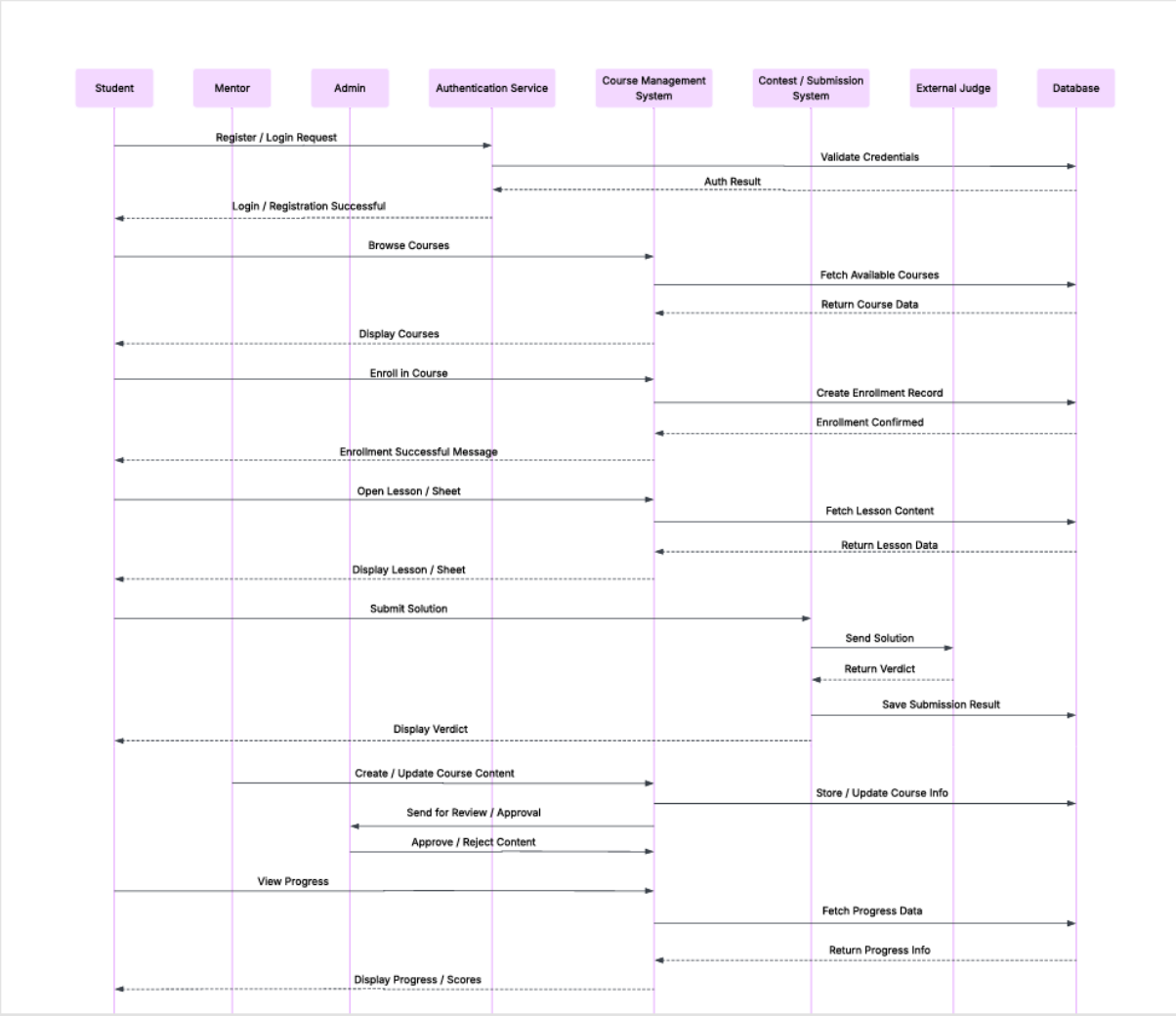


Figure 4.6: System Sequence Diagram.

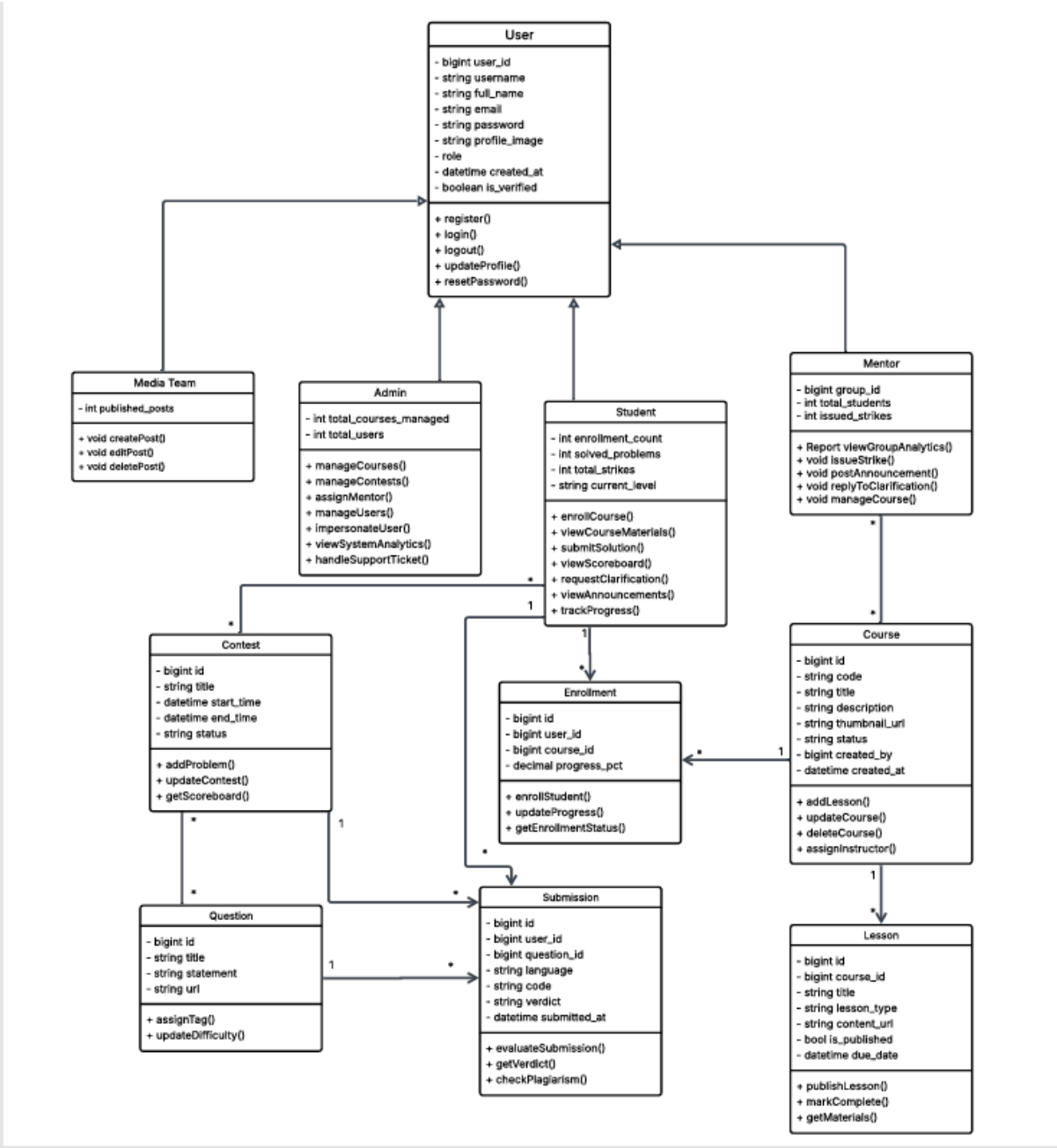


Figure 4.7: System Class Diagram.

Bibliography

[1] Inc. Coursera. Coursera. https://www.coursera.org, 2024.

[2] Inc. 2U. edx. https://www.edx.org, 2024.

[3] beecrowd. beecrowd (formerly uri online judge). https://www.beecrowd.com.br, 2024.

[4] Inc. Respondus. Respondus lockdown browser. https://www.respondus.com, 2024.

[5] Alex Aiken. Moss: A system for detecting software plagiarism. Technical report, University of California, Berkeley, 2000. https://theory.stanford.edu/~aiken/moss/.

[6] Moodle HQ. Moodle - open-source learning platform. https://moodle.org, 2024.

[7] Moodle HQ. Moodle plugins directory. https://moodle.org/plugins, 2024.

[8] Placeholder Author. Moodlejudge: An integration of an online judge into moodle. *Placeholder Journal*, 2010.

[9] Placeholder Author. A survey of computerized adaptive testing. *Placeholder Journal*, 2000.

[10] Placeholder Author. *Computerized Adaptive Testing: A Primer*. Placeholder Publisher, 2010.

[11] Placeholder Author. Adaptive learning and computerized adaptive testing. *Placeholder Journal*, 2018.

[12] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J. Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems (NIPS)*, 2015.

[13] Placeholder Author. Bayesian knowledge tracing. *Placeholder Journal*, 1994.

[14] Placeholder Author. A review of knowledge tracing models. *Placeholder Journal*, 2018.

[15] Placeholder Author. Deep learning for knowledge tracing: A survey. *Placeholder Journal*, 2020.

[16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Nikolay Platonov, Timo Rocktäschel, and Sebastian Riedel. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[17] Placeholder Author. A survey on retrieval-augmented generation. *Placeholder Journal*, 2024.

[18] Placeholder Author. Rag and vector databases. `https://example.com/rag-vector-db`, 2023.

[19] Placeholder Author. Building a chatbot with rag. *Placeholder Journal*, 2023.

[20] Placeholder Author. A review of online proctoring. *Placeholder Journal*, 2019.

[21] Placeholder Author. Online proctoring: A survey of methods and challenges. *Placeholder Journal*, 2021.

[22] Inc. Respondus. Respondus monitor video analysis. `https://www.respondus.com/monitor`, 2024.

[23] Placeholder Author. Anomaly detection in online proctoring. *Placeholder Journal*, 2022.

[24] Placeholder Author. Architectures for online proctoring systems: A survey. *Placeholder Journal*, 2020.

[25] Google. Webrtc project. `https://webrtc.org`, 2024.

[26] Mozilla Developer Network. Real-time communication with webrtc. `https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API`, 2024.

[27] Placeholder Author. A data pipeline for online proctoring. *Placeholder Journal*, 2021.

[28] Placeholder Author. Scalable data processing for proctoring systems. In *Placeholder Conference*, 2022.

[29] Lutz Prechelt, Guido Malpohl, and Michael Philippsen. Jplag: Detecting software plagiarism. *Informatik Spektrum*, 2002.

[30] Placeholder Author. Jplag website. `https://jplag.ipd.kit.edu/`, 2024.

[31] Michael J. Wise. Greedy string tiling algorithm. In *Proceedings of the 20th Annual ACM SIGUCCS Conference on User Services*, 1992.

[32] Placeholder Author. Code obfuscation and its impact on plagiarism detection. *Placeholder Journal*, 2017.

[33] Placeholder Author. Detecting plagiarism using abstract syntax trees. *Placeholder Journal*, 2010.

[34] Placeholder Author. Advanced ast-based plagiarism detection. *Placeholder Journal*, 2015.

[35] Placeholder Author. Machine learning for software plagiarism detection. *Placeholder Journal*, 2019.

[36] Placeholder Author. A survey of machine learning techniques for plagiarism detection. *Placeholder Journal*, 2022.

[37] Placeholder Author. Deep learning approaches for source code plagiarism detection. *Placeholder Journal*, 2023.

[38] Placeholder Author. Introduction to online judges. `https://example.com/oj-intro`, 2021.

[39] Placeholder Author. A survey of online judge systems. *Placeholder Journal*, 2016.

[40] Placeholder Author. Security challenges in online judge systems. *Placeholder Journal*, 2018.

[41] Herman Zvonimir Došilović. Judge0: Open-source online judge. `https://judge0.com/`, 2024.

[42] Herman Zvonimir Došilović. Judge0 api documentation. https://api.judge0.com/, 2024.

[43] Herman Zvonimir Došilović. Judge0 github repository. https://github.com/judge0/judge0, 2024.

[44] Inc. Docker. Docker: Accelerated container application development. https://www.docker.com, 2024.

[45] Placeholder Author. *Docker: Up & Running*. O'Reilly Media, 2018.

[46] Inc. Docker. Docker documentation. https://docs.docker.com, 2024.